

WEEK 9 IMPLEMENTATION REPORT

AI Infrastructure, Spring AI / Ollama & Qwen Model Provisioning

Project Phase:	Week 9 AI Infrastructure	Status:	COMPLETED (94/94 Tests Passed)
Author:	Engineering Architecture Team	Date:	September 24, 2026
AI Infrastructure:	Spring AI 0.8.1, Ollama v0.34.3, Qwen 2.5 Coder 7B	Test Suite:	94 Backend Tests (0 Errors, 0 Failures)

1. Executive Summary & Objective

Week 9 establishes the foundational **AI Backend Infrastructure** for the LearnPulse AI-Powered Learning Platform. This release integrates Spring AI with a locally provisioned **Ollama** LLM instance executing the **Qwen** model (qwen2.5-coder:7b). The primary objective is to deliver a secure, robust, multi-turn AI Tutor service with context management, chat history persistence, and failure isolation: enforcing clean layering (Controller -> AiLlmService -> PromptManager -> ChatClient -> Ollama -> Qwen), failure isolation, PostgreSQL history persistence (ai_chat_history), and JWT RBAC security (ROLE_STUDENT).

```
Student Client (POST /api/ai/chat + JWT Bearer)
  ▼
AI Chat Controller (AiChatController.java)
  ■ (Request Validation @Valid)
  ▼
AI Service Layer (AiLlmService.java)
  ■ (Multi-Turn History Retrieval & Context Isolation)
  ▼
Prompt Management (PromptManager.java)
  ■ (System Prompt + History Messages + User Input)
  ▼
Spring AI ChatClient (AiConfig.java / OllamaChatClient)
  ■ (HTTP RestClient Communication)
  ▼
Local Ollama Service (v0.34.3) ■■■ Qwen Model (qwen2.5-coder:7b)
  ■
  ▼
AI Response Persistence (ai_chat_history PostgreSQL table) ■■■ ApiResponse
```

2. Local AI Environment Provisioning & Verification (Day 49)

Metric / Parameter	Value / Empirical Verification Result
Ollama Service Endpoint	http://localhost:11434
Ollama Version	0.34.3
Active Model / Tag	qwen2.5-coder:7b (7.6B parameters, digest dae161e27b0e...)
Direct Test Prompt	"Explain Java inheritance in two sentences."
Direct Inference Response	"Java inheritance allows a class to inherit attributes and methods from another class, facilitating code reuse and the creation of a class hierarchy."
Ollama Cold Model Load	approximately 44.4 seconds
Warm Direct Inference	approximately 5.77 seconds
Observed Live Multi-Turn API Request	approximately 58.80 seconds
Environment Status	PROVISIONED & ACTIVE

Note: Local inference latency is hardware/model dependent.

3. Spring AI Integration & Architecture (Day 50 & 51)

- **Dependencies (pom.xml):** Integrated spring-ai-bom version 0.8.1 and spring-ai-ollama-spring-boot-starter with Spring Boot 3.2.5.
- **Application Properties (application.yml):** Externalized spring.ai.ollama.base-url ({OLLAMA_BASE_URL:http://localhost:11434}) and spring.ai.ollama.chat.options.model ({OLLAMA_MODEL:qwen2.5-coder:7b}).
- **ChatClient Configuration (AiConfig.java):** Configured OllamaApi and OllamaChatClient bean exposed via ChatClient interface with @Primary.
- **Service Layer Isolation (AiLlmService.java):** Controller delegates strictly to AiLlmService. Handles LLM calls, prompt building, multi-turn history retrieval, DB persistence, and graceful fallback handling.
- **Failure Isolation:** Ollama connection failures are caught safely, returning controlled fallback responses (fallback: true) without crashing the LMS or exposing raw stack traces.

4. Prompt Management Architecture (Day 52)

- **PromptManager (PromptManager.java):** Separates System instructions from User input. Configures the Academic Tutor persona (supportive, clear, educational context, concise, asks for clarification when unclear, avoids unsupported claims).
- **Context & Layering:** Supports dynamic subject focus injection (subjectName) and layers SystemMessage, prior turns (UserMessage/AssistantMessage), and current UserMessage into a Spring AI Prompt object.

5. AI Tutor API & Security (Day 53)

- **API Endpoint:** POST /api/ai/chat protected by JWT Spring Security requiring ROLE_STUDENT.
- **DTOs:** AiChatRequest (@NotBlank, @Size(max=2000) message, conversationId, subjectName) and AiChatResponse (conversationId, userMessage, aiResponse, timestamp, fallback).
- **PostgreSQL Persistence & Student Isolation:** AiChatHistory entity (ai_chat_history table) saving user_id, conversation_id, user_message, ai_response, created_at. Enforces Student Conversation Isolation via user_id + conversation_id filtering.
- **Multi-Turn Context:** Injects up to last 10 historical conversation turns for (userId, conversationId) to provide context continuity.

6. AI Framework Initialization & Integration Test Results (Day 54)

Test ID	Test Scenario	Request / Setup	Expected Result	Actual Result	Status
AI-001	Basic AI Chat Request	Auth Student, valid query	HTTP 200, valid AI response	HTTP 200, valid text	PASS
AI-002	Input Validation Rejection	Blank message " "	HTTP 400 Bad Request	HTTP 400, validation error	PASS
AI-003	Ollama Service Fallback	Connection exception thrown	HTTP 200, fallback: true	HTTP 200, controlled message	PASS
AI-004	Multi-Turn Conversation	Existing conversation ID	HTTP 200, history in prompt	HTTP 200, context injected	PASS
AI-005	Basic Sequential Stability Testing	Sequential chat requests (no concurrency)	HTTP 200, stable responses	HTTP 200, all turns passed	PASS
AI-006	Chat History Persistence	Successful chat invocation	Record saved in PostgreSQL	Record persisted in DB	PASS
AI-007	Unauthenticated Rejection	No JWT Bearer header	HTTP 401 Unauthorized	HTTP 401 Unauthorized	PASS
AI-008	Role Guard Rejection	Authenticated Teacher token	HTTP 403 Forbidden	HTTP 403 Forbidden	PASS
AI-009	Large Input Validation	Message > 2000 chars	HTTP 400 Bad Request	HTTP 400, size error	PASS
AI-010	Standard ApiResponse Format	Valid chat request	Standard response wrapper	Status success & timestamp	PASS

7. Empirical Live Verification & Basic Sequential Stability Results

- **Live Endpoint Verification:** Verified POST `/api/ai/chat` against running Spring Boot server on port 8080 communicating with local Ollama 0.34.3 and qwen2.5-coder:7b.
- **Student Auth & Multi-Turn Context:** Executed registration, login, Turn 1 query, and Turn 2 follow-up query maintaining multi-turn context across turns.
- **Basic Sequential Stability Testing:** Evaluated basic sequential stability using consecutive chat requests. No concurrent load testing was performed.
- **Local Inference Performance Metrics:** Recorded Ollama cold model load (~44.4 seconds), warm direct inference (~5.77 seconds), and observed live multi-turn API request (~58.80 seconds). Note: Local inference latency is hardware/model dependent.

8. Summary of Files & Modifications

Files Added (9 total):

1. `src/main/java/com/learnpulse/backend/entity/AiChatHistory.java` – Chat history entity
2. `src/main/java/com/learnpulse/backend/repository/AiChatHistoryRepository.java` – Chat history JPA repository
3. `src/main/java/com/learnpulse/backend/dto/AiChatRequest.java` – Request DTO with Bean Validation
4. `src/main/java/com/learnpulse/backend/dto/AiChatResponse.java` – Response DTO
5. `src/main/java/com/learnpulse/backend/ai/PromptManager.java` – System prompt & template builder
6. `src/main/java/com/learnpulse/backend/config/AiConfig.java` – Spring AI ChatClient bean configuration
7. `src/main/java/com/learnpulse/backend/service/AiLlmService.java` – LLM service & fallback handler
8. `src/main/java/com/learnpulse/backend/controller/AiChatController.java` – REST controller for POST `/api/ai/chat`

9. src/test/java/com/learnpulse/backend/ai/AiChatIntegrationTest.java – Automated integration test suite

Files Modified (3 total):

- 1. pom.xml – Added Spring AI BOM (0.8.1), spring-ai-ollama-spring-boot-starter, & Spring Milestones repo
- 2. src/main/resources/application.yml – Added spring.ai.ollama configuration section
- 3. src/main/java/com/learnpulse/backend/security/SecurityConfig.java – Added /api/ai/** security matcher requiring ROLE_STUDENT

9. Scope Exclusions & Known Limitations

- **Scope Exclusions Enforced:** No RAG, no vector search, no pgvector embeddings, no document retrieval, no AI quiz generation, no frontend UI modifications.
- **Git Operations:** No git commit or git push commands were executed.
- **Architectural Integrity:** Week 1-8 Spring Security, JWT authentication, and database schemas were 100% preserved.

10. Sign-Off & Implementation Status

Verification Gate	Status	Empirical Results & Details
Backend Integration Tests	PASSED	94/94 tests passed (mvn test BUILD SUCCESS)
Local Ollama & Qwen Model	VERIFIED	Ollama v0.34.3 + qwen2.5-coder:7b active & responsive
Live Endpoint Verification	VERIFIED	POST /api/ai/chat verified for Turn 1 & Turn 2 multi-turn context
Failure Isolation	VERIFIED	Controlled fallback response returned without crashing LMS
Mentor Requirements	VERIFIED	Implemented and verified against the Week 9 specification

Mentor Requirements: Implemented and verified against the Week 9 specification